



Universidade do Minho

Braga, Portugal

CONCEÇÃO DE MODELOS DE APRENDIZAGEM E DECISÃO

RELATÓRIO DO TRABALHO PRÁTICO

Aprendizagem e Decisão Inteligentes

Departamento de Informática

Engenharia Informática 2025/26

Grupo 17:

A106936 - Duarte Escairo Brandão Reis Silva

A106932 - Luís António Peixoto Soares

A106856 - Tiago Silva Figueiredo

A104704 - Inês Ferreira Ribeiro

16 Maio 2026

Índice

1. Introdução	1
2. Metodologia	1
3. Dataset atribuído	2
3.1. Exploração do dataset	2
3.1.1. Análise da variável temporal	2
3.1.2. Variáveis de consumo energético (Voltage)	2
3.1.3. Colunas meteorológicas	3
3.1.4. Coluna alvo (Target)	5
3.2. Tratamento de dados	5
3.2.1. Tratamento da data e criação de colunas temporais	5
3.2.2. Tratamento da coluna alvo Consumptions	6
3.2.3. Tratamento da coluna snowfall	6
3.2.4. Tratamento das colunas rain e precipitation	6
3.2.5. Tratamento da terrestrial_radiation	6
3.2.6. Tratamento das colunas Diffuse_Radiation e Direct_Radiation	7
3.2.7. Tratamento das colunas de tensão (Voltage)	8
3.2.8. Tratamento das colunas de vento	8
3.2.9. Tratamento das variáveis atmosféricas (temperatura, humidade, pressão e cloud cover)	9
3.3. Modelos desenvolvidos	10
3.3.1. Decision Tree Learner	10
3.3.2. Random Forest Learner	11
3.3.3. Logistic Regression Learner	11
3.3.4. Gradient Boosted Trees Learner	11
3.3.5. RProp MLP Learner (Redes Neuronais)	12
3.3.6. Modelos de Clustering	13
3.4. Resultados obtidos e discussão	13
3.4.1. Impacto do tratamento de dados nos resultados	14
3.4.2. Decision Tree	15
3.4.3. Random Forest	15
3.4.4. Logistic Regression	16
3.4.5. Gradient Boosted Trees	16
3.4.6. Redes Neuronais (RProp MLP)	16
3.4.7. Clustering	17
3.4.8. Partição simples vs. validação cruzada	17
3.4.9. Sumário e melhores resultados	18
4. Dataset de grupo	18
4.1. Exploração do dataset	18
4.1.1. Variáveis discretas	19
4.1.2. Variáveis contínuas	19
4.1.3. Correlação entre variáveis	20
4.2. Tratamento de dados	21
4.3. Modelos desenvolvidos	22
4.3.1. Validação	22
4.3.2. Linear Regression	23
4.3.3. Polynomial Regression	24

4.3.4. Random Forest	24
4.3.5. Gradient Boosted Trees	24
4.3.6. Redes Neurais Artificiais	24
4.4. Discussão dos resultados obtidos	25
4.4.1. Melhores resultados	25
4.4.2. Piores resultados	26
4.4.3. Comparação entre modelos	26
4.4.4. Influência da parametrização dos modelos	27
4.4.5. Influência do tratamento de dados	27
5. Conclusão	29

1. Introdução

O presente relatório foi elaborado no âmbito da unidade curricular de Aprendizagem e Decisão Inteligentes (ADI). O trabalho desenvolvido enquadra-se num projeto de grupo que visa a conceção, implementação e avaliação de modelos de Machine Learning aplicados a dois conjuntos de dados distintos.

O trabalho encontra-se organizado em torno de duas tarefas independentes. A primeira, designada Tarefa Dataset Atribuído, incide sobre um conjunto de dados de consumo energético elétrico, que regista medições horárias de consumo em diferentes níveis de tensão, complementadas por variáveis meteorológicas. A segunda tarefa, designada Tarefa Dataset Grupo, incide sobre um dataset selecionado pela equipa, o Student Performance Factors.

Os principais objetivos deste trabalho foram: compreender e explorar os dados de ambos os datasets, realizar o tratamento e preparação necessários, conceber e otimizar modelos de Machine Learning de classificação, regressão e, no caso do dataset de consumo energético, de clustering; e, por fim, analisar criticamente os resultados obtidos, identificando os modelos mais adequados para cada problema.

Todo o desenvolvimento prático foi realizado com recurso à plataforma **KNIME**, uma ferramenta visual de fluxo de trabalho orientada a dados que permite construir pipelines completos de análise sem necessidade de programação explícita, através da composição de nós configuráveis.

2. Metodologia

Para orientar o processo de análise e desenvolvimento dos modelos, foi adotada a metodologia **SEMMA** (Sample, Explore, Modify, Model and Assess), proposta pelo SAS Institute, que define um conjunto de etapas sequenciais para a condução de projetos de descoberta de conhecimento em dados.

A fase de amostragem (Sample) correspondeu à identificação e seleção dos dados a utilizar. No dataset de consumo energético, foram consideradas as 8736 observações horárias disponíveis na sua totalidade, dado que o volume de dados se revelou adequado para o treino e teste dos modelos sem necessidade de redução. No dataset de desempenho estudantil, procedeu-se de forma semelhante, utilizando os 6607 registos disponíveis.

A fase de exploração (Explore) foi aplicada a ambos os datasets com o objetivo de compreender a natureza das variáveis, as suas distribuições, e as relações e correlações existentes entre elas. Esta fase incluiu a produção de estatísticas descritivas, visualizações de distribuições e matrizes de correlação, suportadas pelos nós de análise estatística do KNIME.

A fase de modificação (Modify) englobou o tratamento dos dados e a preparação das variáveis para os modelos. Em ambos os datasets foram identificados e tratados valores em falta (missing values), recorrendo a estratégias de tratamento ou eliminação conforme adequado. Foram ainda realizadas operações de normalização de variáveis numéricas, codificação de variáveis categóricas, e seleção de atributos relevantes. No dataset de consumo, esta fase incluiu também a criação de novas variáveis de forma a enriquecer a representação dos padrões de consumo.

A fase de modelação (Model) consistiu na implementação e configuração dos modelos de Machine Learning. Neste trabalho foram utilizadas técnicas do paradigma de aprendizagem supervisionada - nomeadamente técnicas de classificação e de regressão — e, no dataset de consumo energético, também técnicas do paradigma de aprendizagem não supervisionada através de modelos de clustering. Na aprendizagem supervisionada, a existência de uma variável alvo conhecida permite ao modelo aprender a partir de resultados pretendidos; na aprendizagem não supervisionada, os algoritmos de clustering procuram identificar agrupamentos naturais nos dados sem recurso a qualquer resultado pretendido.

A fase de avaliação (Assess) correspondeu à análise dos resultados obtidos por cada modelo, com recurso a métricas adequadas a cada paradigma. Os resultados foram comparados entre modelos e interpretados no contexto dos problemas em análise, culminando nas conclusões e recomendações apresentadas nas secções correspondentes deste relatório.

3. Dataset atribuído

3.1. Exploração do dataset

O dataset atribuído diz respeito ao consumo de energia elétrica, registado com uma granularidade horária, e é composto por 8736 registos e 24 colunas (variáveis). Para além das medições de consumo propriamente ditas, o dataset inclui um conjunto alargado de variáveis meteorológicas que caracterizam as condições ambientais em cada momento de registo. A variável alvo, denominada **Consumptions**, classifica o nível de consumo em cinco categorias ordinais: **Low Consumption**, **Medium-Low Consumption**, **Medium Consumption**, **Medium-High Consumption** e **High Consumption**.

A fase de exploração foi feita antes de qualquer intervenção nos dados, com o objetivo de perceber o comportamento de cada variável, identificar problemas de qualidade de dados e perceber as relações existentes entre os campos.

3.1.1. Análise da variável temporal

A coluna **Date/Time** é lida como uma String no formato AAAA-MM-DD HH:MM:SS, sem valores em falta nem registos duplicados. A análise revelou que os campos dos minutos e dos segundos assumem sempre o valor zero, confirmando que o intervalo dos registos é de uma hora. Verificou-se ainda que os dados cobrem múltiplos anos, embora com distribuição irregular: apenas o ano de 2025 apresenta cobertura completa e consistente, o que levou à decisão de filtrar o dataset para esse ano durante o tratamento.

3.1.2. Variáveis de consumo energético (Voltage)

As colunas referentes ao consumo nos diferentes níveis de tensão — **Very High Voltage**, **High Voltage**, **Medium Voltage**, **Low Voltage** e **Total** - apresentam comportamentos distintos. **Very High Voltage**, **High Voltage** e **Low Voltage** são lidas como valores numéricos do tipo Float, enquanto **Medium Voltage** e **Total** são lidas como String, o que indicou a necessidade de conversão de tipo.

Todas as colunas de tensão apresentam valores em falta: 919 em **Very High Voltage**, 844 em **High Voltage** e 875 em **Low Voltage**. As distribuições destas colunas são aproximadamente normais.

A coluna **Total** é derivada da soma das restantes, pelo que os seus valores em falta podem ser recalculados a partir das componentes individuais. Esta relação matemática entre colunas foi importante para definir a estratégia do tratamento de dados.

3.1.3. Colunas meteorológicas

As variáveis meteorológicas apresentam padrões de valores em falta e distribuições variadas. A **temperature_2m** e a **apparent_temperature** apresentam, cada uma, cerca de 830–845 valores em falta, com distribuições aproximadamente normais e gamas entre $2,1^{\circ}\text{C}$ e $43,2^{\circ}\text{C}$ para a temperatura real, e entre $-1,4^{\circ}\text{C}$ e 43°C para a temperatura aparente. A análise de correlação revelou uma correlação praticamente perfeita (próxima de +1) entre estas duas colunas, o que constitui um forte indício de redundância de informação.

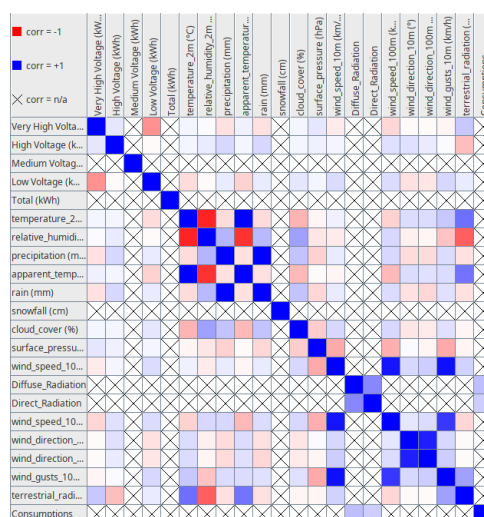


Figura 1: Matriz das correlações lineares

As colunas **precipitation** e **rain** apresentam a maior proporção de valores em falta de todo o dataset: 7747 e 7755 registos, respetivamente, de um total de 8736. As distribuições de ambas são fortemente assimétricas, com a grande maioria dos registos existentes próximos de zero, e a análise de correlação revelou uma correlação também próxima de +1 entre elas, evidenciando uma nova redundância.

A coluna **snowfall** apresenta valores em falta (841), mas, de forma relevante, todos os valores existentes são zero - ou seja, a coluna não transporta qualquer informação para o problema em análise.

A **cloud_cover** apresenta 885 valores em falta e uma distribuição com concentrações elevadas tanto próximas de 0% como de 100%.

A **surface_pressure** distribui-se numa gama de valores estreita entre 965,8 e 1004,7 hPa, e apresenta cerca de 874 valores em falta.

As variáveis de vento - `wind_speed_10m`, `wind_speed_100m` e `wind_gusts_10m` - apresentam distribuições assimétricas à esquerda, com a maioria dos registos abaixo da média.

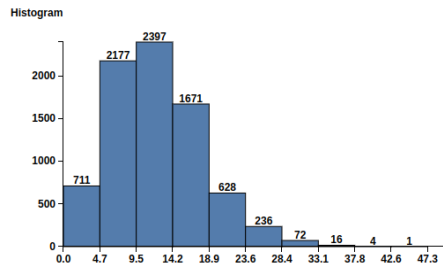


Figura 2: Distribuição `wind_speed_10m`

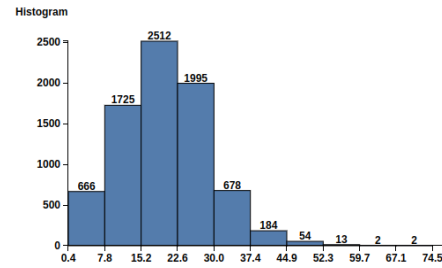


Figura 3: Distribuição `wind_speed_100m`

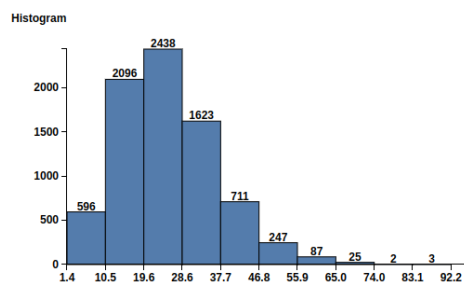
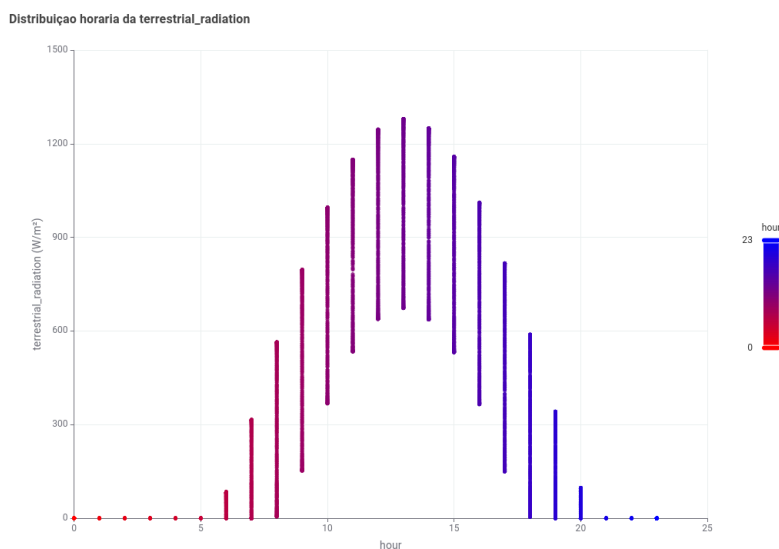


Figura 4: Distribuição `wind_gusts_10m`

As colunas de direção do vento (`wind_direction_10m` e `wind_direction_100m`) apresentam distribuições quase lineares, com ligeira concentração para valores superiores, e ambas têm cerca de 880–895 valores em falta. A correlação entre as duas medições de direção é muito elevada, dado que uma mede a direção a 10 metros e a outra a 100 metros de altitude, capturando o mesmo fenómeno em altitudes diferentes.

A `terrestrial_radiation` apresenta 869 valores em falta e uma distribuição assimétrica com cerca de 50% dos registos entre 0 e 128 W/m^2 , o que é fisicamente coerente: durante as horas noturnas, a radiação terrestre é nula, pelo que esse grupo de valores baixos reflete simplesmente os períodos sem luz solar.



As colunas `Diffuse_Radiation` e `Direct_Radiation` são variáveis qualitativas ordinais, armazenadas como `String`. `Diffuse_Radiation` apresenta três categorias (None, Low, Medium) e `Direct_Radiation` quatro (None, Low, Medium, High), com 891 e 863 valores em falta, respetivamente. À semelhança da `terrestrial_radiation` a distribuição destas variáveis poderá estar relacionada com a hora do dia e com a estação do ano.

3.1.4. Coluna alvo (Target)

A coluna `Consumptions` não apresenta valores em falta, mas revelou um problema importante de consistência dos dados: existem 13 categorias distintas, quando deveriam existir apenas 5. A análise identificou algumas variações (Lw, Med, Hgh, MedLow, MedHgh, MedioBaixo, MedioAlto), que representam ruído introduzido na recolha dos dados. Estas variações representam um número muito reduzido de registos (entre 2 e 5 por categoria incorreta), mas a sua correção é essencial para que o classificador identifique corretamente as cinco classes alvo.

A distribuição das cinco classes corretas mostra algum desequilíbrio: High Consumption é a classe minoritária com 788 registos, enquanto Medium-High Consumption é a maioritária com 2173. As restantes três classes distribuem-se de forma mais equilibrada entre si, com valores entre 1671 e 2085 registos.

3.2. Tratamento de dados

O tratamento de dados foi concebido para resolver todos os problemas identificados na fase de exploração, garantindo a qualidade e coerência do dataset antes da aplicação dos modelos. Dada a complexidade e diversidade dos problemas encontrados - em particular a variedade de tipos de valores em falta e as relações entre colunas - foram desenvolvidas várias abordagens de tratamento, organizadas num fluxo principal e em cinco variantes. Cada variante parte do fluxo principal e difere deste numa ou mais decisões de tratamento específicas, permitindo avaliar o impacto dessas escolhas no desempenho dos modelos.

3.2.1. Tratamento da data e criação de colunas temporais

A coluna `Date/Time`, lida como `String`, foi convertida para o tipo `Date&Time` nativo do KNIME através do nó **String to Date&Time**, com o formato AAAA-MM-DD HH:MM:SS. De seguida, um nó **Row Filter** foi utilizado para retirar linhas com datas inválidas - designadamente registos fora do intervalo esperado ou com formato incorreto. O nó **Date&Time Part Extractor** decompôs então a coluna temporal nos seus constituintes: ano, mês, dia do ano, dia da semana e hora, criando cinco novas colunas numéricas que representam explicitamente os padrões do consumo.

O dataset foi subsequentemente ordenado cronologicamente com um nó **Sorter** e filtrado com um novo **Row Filter** para reter apenas os registos do ano de 2025, já que a esmagadora maioria dos registos eram referentes a esse ano. A coluna do ano foi depois removida com **Column Filter**, por ser constante após o filtro. Foram ainda criadas duas colunas derivadas através de dois nós **Rule Engine**: a estação do ano (`estacao_ano`), determinada pelo mês - Primavera de março a maio, Verão de junho a agosto, Outono de setembro a novembro e Inverno nos restantes meses - e uma variável booleana `weekend`, que indica se o dia corresponde a um fim de semana (dia da semana igual ou superior a 6).

Estas variáveis enriquecem a representação temporal do consumo, capturando padrões sazonais e de comportamento semanal que não estavam implícitos no timestamp original.

3.2.2. Tratamento da coluna alvo **Consumptions**

A correção das inconsistências da variável alvo foi realizada com um nó **Rule Engine**, através de um conjunto de regras que mapeiam cada variação incorreta para a categoria canónica correspondente: Lw para Low Consumption; MedLow e MedioBaixo para Medium-Low Consumption; Med e Medium para Medium Consumption; MedHgh e MedioAlto para Medium-High Consumption; e Hgh para High Consumption. Todos os restantes valores são preservados sem alteração.

3.2.3. Tratamento da coluna **snowfall**

Dado que todos os valores existentes na coluna **snowfall** eram zero, esta coluna foi simplesmente eliminada através de um nó **Column Filter**. A inclusão desta coluna no dataset não traria qualquer informação adicional por se tratar de uma coluna constante, podendo inclusive introduzir ruído.

3.2.4. Tratamento das colunas **rain** e **precipitation**

Como demonstrado na fase de exploração, **rain** e **precipitation** medem o mesmo fenómeno e apresentam uma correlação praticamente perfeita. A estratégia adotada em todas as variantes foi a fusão das duas colunas numa única, através de um nó **Column Merger**, que preenche os valores em falta de uma coluna com os valores da outra. Após a fusão, os valores ainda em falta em **precipitation** foram substituídos por zero com um nó **Missing Value**, o que pode ser justificado pelo facto da grande maioria dos valores existentes ser muito próximo de zero.

3.2.5. Tratamento da **terrestrial_radiation**

Esta coluna foi objeto de uma das diferenças mais relevantes entre o fluxo principal e as variantes. No fluxo principal e na maioria das variantes, optou-se por um tratamento baseado em conhecimento de domínio: dado que a radiação terrestre segue um padrão determinístico, é nula durante a noite e segue uma curva previsível ao longo do dia, com valores dependentes da estação do ano - foi construída uma tabela de referência com os valores medianos de radiação para cada combinação de hora do dia e estação do ano.

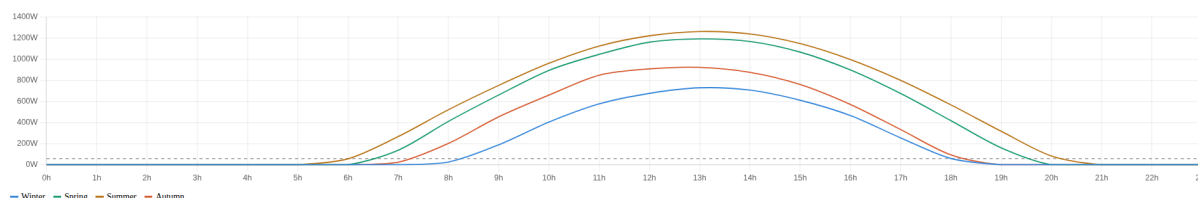


Figura 6: Distribuição de **terrestrial_radation** conforme a estação do ano e hora

MEDIANAS DE `terrestrial_radiation` (W/m²) por hora × estação

Hora	Winter	Spring	Summer	Autumn
h=00	0	0	0	0
h=01	0	0	0	0
h=02	0	0	0	0
h=03	0	0	0	0
h=04	0	0	0	0
h=05	0	0	0	0
h=06	0	0	55.9	0
h=07	0	137.2	266.8	23.7
h=08	25.2	410.1	520.5	201.8
h=09	188.5	659.7	750.7	452.9
h=10	403.6	892.8	960.0	658.8
h=11	575.6	1044.4	1122.8	847.3
h=12	675.2	1159.5	1219.6	906.8
h=13	727.7	1190.5	1260.2	921.0
h=14	705.7	1166.1	1237.7	874.4
h=15	610.0	1065.4	1146.6	759.1
h=16	465.4	896.8	995.5	570.3
h=17	254.0	673.9	798.4	333.1
h=18	58.9	417.1	566.1	93.5
h=19	0	160.1	317.1	0
h=20	0	0	82.7	0
h=21	0	0	0	0
h=22	0	0	0	0
h=23	0	0	0	0

Figura 7: Distribuição dos valores de `terrestrial_radiation` conforme a estação do ano e hora, que deu origem à tabela.

Esta tabela foi codificada num nó **Rule Engine** com 96 regras (24 horas × 4 estações), produzindo uma coluna de previsão que foi depois fundida com a coluna original através de um **Column Merger**, preenchendo apenas os valores em falta. Numa das variantes do tratamento de dados, optou-se em vez disso pela interpolação linear com um nó **Missing Value**, sem recorrer à informação temporal e sazonal. Esta diferença permite avaliar se o conhecimento de domínio incorporado no tratamento baseado em regras produz melhores resultados do que um tratamento puramente estatístico.

3.2.6. Tratamento das colunas `Diffuse_Radiation` e `Direct_Radiation`

Em consonância com a abordagem adotada para a `terrestrial_radiation`, os valores em falta de `Diffuse_Radiation` e `Direct_Radiation` foram tratados com base em regras derivadas do comportamento físico da radiação solar.

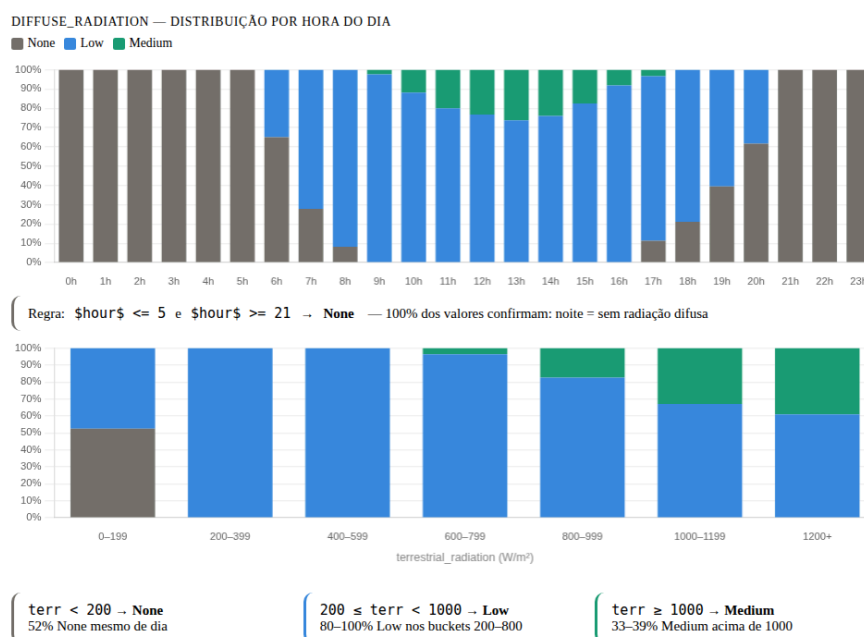


Figura 8: Distribuição de `Diffuse_Radiation` por hora e `terrestrial_radiation`

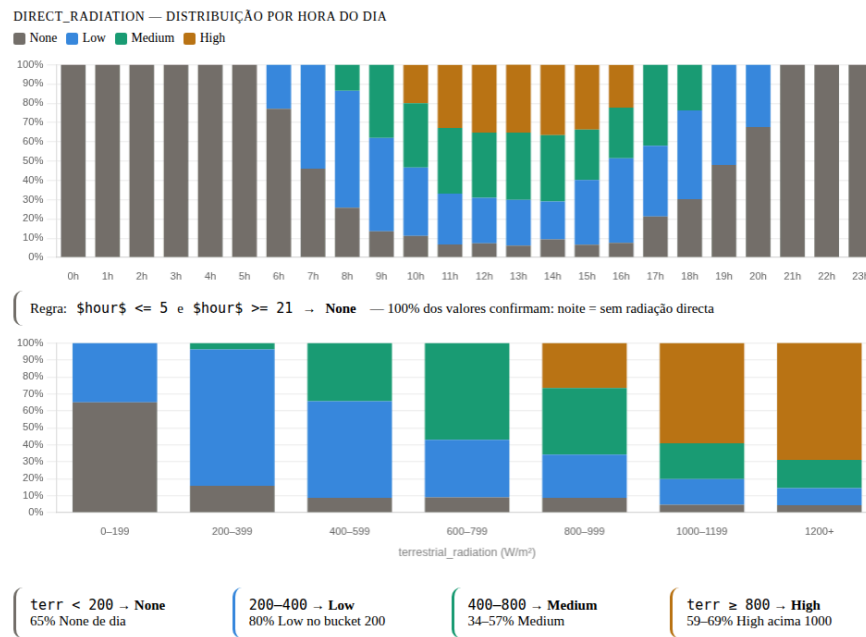


Figura 9: Distribuição de Direct_Radation por hora e terrestrial_radiation

Para **Diffuse_Radiation**, a regra atribui **None** às horas noturnas (hora ≤ 5 ou hora ≥ 21), **None** quando a radiação terrestre é inferior a 200 W/m², **Low** quando inferior a 1000 W/m², e **Medium** nos restantes casos. Para **Direct_Radiation**, acrescenta-se um limiar de **High** para valores de radiação terrestre superiores a 800 W/m². Os valores tratados foram fundidos com os originais através de dois nós **Column Merger**.

3.2.7. Tratamento das colunas de tensão (Voltage)

As colunas **Medium Voltage** e **Total**, lidas como **String**, foram convertidas para **Number (Double)** através de um nó **String to Number**. O tratamento dos valores em falta nas cinco colunas de tensão constitui uma segunda diferença relevante entre as variantes: no fluxo principal e em algumas variantes, foi usada interpolação linear (nó **Missing Value** com estratégia **Linear Interpolation**), que aproveita a continuidade temporal das medições para estimar os valores em falta com base nos registos vizinhos. Noutras variantes, optou-se pela substituição com a mediana. Após o preenchimento dos valores em falta, a coluna **Total** foi recalculada em todos os fluxos através de um nó **Math Formula**, somando as quatro componentes de tensão, garantindo a consistência matemática do dataset.

3.2.8. Tratamento das colunas de vento

As variáveis de velocidade do vento (**wind_speed_10m**, **wind_speed_100m** e **wind_gusts_10m**) e de direção (**wind_direction_10m**, **wind_direction_100m**) apresentam diferentes estratégias de tratamento entre variantes. No fluxo principal e outras variantes, as colunas de velocidade e rajadas (**gusts**) foram tratadas com interpolação linear, enquanto as colunas de direção receberam um tratamento especial: foram primeiro submetidas a um **Column Merger** cruzado - fundindo **wind_direction_10m** com **wind_direction_100m** e vice-versa - para preencher valores em falta de uma com os da outra quando disponíveis, aproveitando a alta correlação entre ambas.

Os valores em falta foram depois tratados com uma média móvel de passo 2 (Moving Average no nó **Missing Value**), que é uma estratégia mais adequada do que o valor fixo da média/mediana, já que acompanha melhor os valores, apesar de em casos raros poder derivar um valor longe do correto por se tratarem de ângulos. Noutras variantes, optou-se pela substituição simples com a mediana para todas as colunas de vento, sem distinção entre velocidade e direção.

3.2.9. Tratamento das variáveis atmosféricas (temperatura, humidade, pressão e cloud cover)

A fusão das colunas `temperature_2m` e `apparent_temperature` numa única coluna de temperatura foi realizada em quase todos os fluxos, através de um nó **Column Merger**. Esta decisão é justificada pela correlação quase perfeita entre as duas colunas: manter ambas introduziria redundância sem benefício para os modelos.

Para os valores em falta de temperatura, humidade e pressão, a maioria das variantes usaram interpolação linear, dado que estas são séries temporais com variação gradual e contínua. A exceção foi uma das variantes, que utilizou a mediana como estratégia de tratamento para estas colunas.

A coluna `cloud_cover` apresentou um desafio específico: a sua distribuição particular tornava pouco adequada um tratamento simples por interpolação ou mediana global. A solução adotada foi calcular a mediana de `cloud_cover` por estação do ano — 91% no Inverno, 42% na Primavera, 29% no Outono e 0% no Verão — e usar estes valores, gerados por um nó **Rule Engine**, para preencher os valores em falta através de um **Column Merger**. Esta abordagem captura a sazonalidade da cobertura de nuvens e é mais informativa do que uma mediana global.

O quadro seguinte resume as principais diferenças entre os fluxos de tratamento:

Coluna	Fluxo principal	V.Vento	V.Voltage	V.Terrestrial	V. Atm. com Mediana	V. Atm. com Merge
Terrestrial rad.	Regras	Regras	Regras	Interp. linear	Regras	Interp. linear
Tensões (voltage)	Interp. linear	Interp. linear	Mediana	Mediana	Interp. linear	Interp. linear
Vento (speed/gusts)	Interp. linear	Mediana	Mediana	Mediana	Interp. linear	Interp. linear
Vento (direção)	Moving average	Mediana	Mediana	Mediana	Moving average	Moving average
Temperatura	Interp. linear	Interp. linear	Interp. linear	Interp. linear	Mediana	Interp. linear
Temp. aparente	Separada	Fundida	Fundida	Fundida	Separada	Fundida

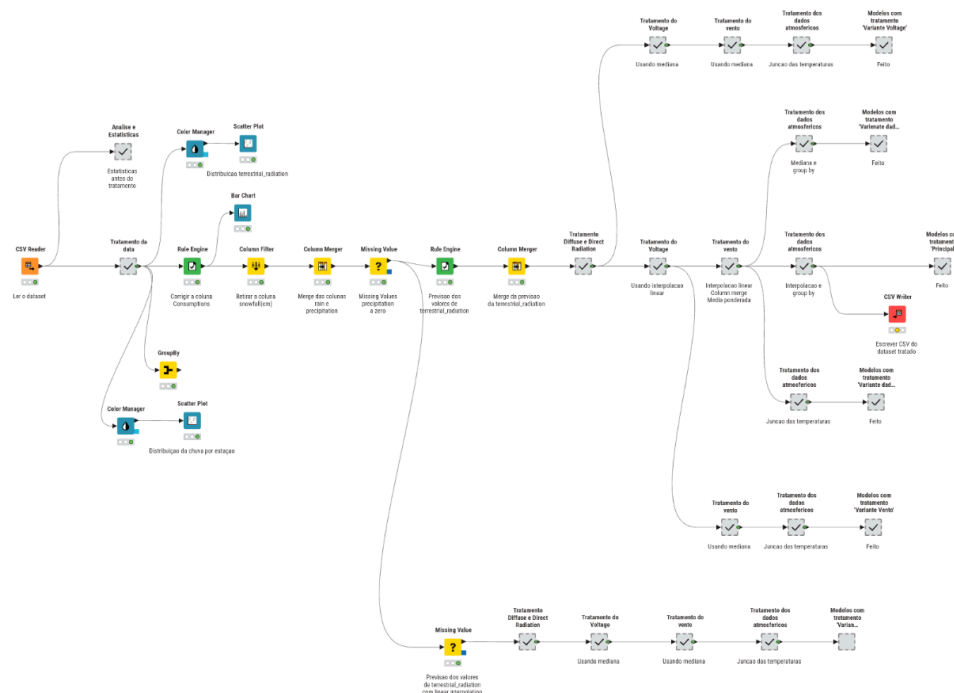


Figura 10: Vista geral da organização do workflow do dataset atribuído

3.3. Modelos desenvolvidos

Após o tratamento de dados, foram desenvolvidos e avaliados modelos de classificação e de clustering, aplicados ao dataset preparado pelas diferentes variantes de tratamento. A avaliação foi conduzida com duas estratégias de validação distintas: divisão simples treino/teste com partição 70%/30% através do nó **Table Partitioner**, e validação cruzada com 10 iterações de amostragem aleatória através do par de nós **X-Partitioner / X-Aggregator**. Para os modelos de classificação (paradigma supervisionado) e para os modelos de clustering (paradigma não supervisionado), a métrica principal usada foi a accuracy (percentagem de instâncias corretamente classificadas).

3.3.1. Decision Tree Learner

A árvore de decisão é um dos algoritmos mais intuitivos de Machine Learning: o modelo aprende a fazer perguntas sobre os dados - por exemplo, “a temperatura é superior a 25°C?” - e vai dividindo os registos em grupos cada vez mais homogêneos, até conseguir atribuir uma classe a cada grupo. O resultado é uma estrutura em árvore que pode ser visualizada e interpretada diretamente.

No KNIME, este modelo é implementado pelo nó **Decision Tree Learner**. Foram testados dois critérios que determinam como cada divisão é feita: o **Gini Index**, que mede o grau de mistura de classes em cada grupo, e o **Gain Ratio**, que pondera o ganho de informação de cada divisão pelo número de grupos que ela cria, penalizando divisões que gerem demasiados grupos pequenos.

Para cada critério foram testadas duas configurações: com e sem *pruning* MDL. O *pruning* (poda) é um processo aplicado após a construção da árvore que remove ramos

que complicam o modelo sem melhorar significativamente a sua precisão — funciona como uma simplificação que ajuda o modelo a generalizar melhor para dados novos. Sem *pruning*, a árvore cresce livremente e tende a memorizar os dados de treino, o que piora o desempenho em dados não vistos.

3.3.2. Random Forest Learner

A floresta aleatória é uma extensão natural da árvore de decisão: em vez de construir uma única árvore, constroem-se várias neste caso foram testadas configurações com 100 e 200 árvores (modelos) - e a previsão final resulta da votação maioritária entre todas elas. A ideia central é que um conjunto de modelos que erram de forma diferente produz, em conjunto, um resultado mais fiável do que qualquer modelo individual.

Este modelo é implementado pelo nó **Random Forest Learner** no KNIME. Para garantir que as árvores são suficientemente diferentes entre si, o algoritmo introduz aleatoriedade em dois momentos: seleciona uma subamostra aleatória dos dados de treino para cada árvore (*bagging*), e em cada divisão considera apenas um subconjunto aleatório dos atributos disponíveis.

Foram testados os mesmos critérios de divisão da árvore simples — **Gini**, **Information Gain** e **Information Gain Ratio** — sendo o **Information Gain** o ganho de entropia clássico e o **Information Gain Ratio** a sua variante que penaliza divisões com muitos grupos, tal como o **Gain Ratio**.

3.3.3. Logistic Regression Learner

A regressão logística é um modelo de classificação que estima a probabilidade de cada registo pertencer a cada classe, com base numa combinação linear dos atributos. Apesar do nome incluir “regressão”, trata-se de um classificador. Como a variável alvo tem cinco classes, o modelo usa regressão logística multinomial, treinando internamente um classificador por classe.

O nó utilizado é o **Logistic Regression Learner**. Foram testados dois métodos de otimização: o **IRLS** (Iteratively Reweighted Least Squares), um método mais preciso mas mais lento, e o **SAG** (Stochastic Average Gradient), mais rápido em datasets grandes mas que exige que os dados estejam normalizados para funcionar bem. Por isso, para o SAG foram sempre usadas configurações com normalização; para o IRLS foram testadas versões com e sem normalização. Foram ainda comparadas duas versões dos dados de entrada: uma com todos os campos disponíveis, incluindo variáveis categóricas, e outra apenas com os campos numéricos (*float*), uma vez que a regressão logística não processa nativamente variáveis categóricas sem codificação prévia.

3.3.4. Gradient Boosted Trees Learner

O gradient boosting é outro método de *ensemble*, mas com uma filosofia diferente da floresta aleatória. Aqui as árvores são construídas de forma sequencial: cada nova árvore é treinada para corrigir os erros que a árvore anterior cometeu. O resultado é um modelo que vai melhorando progressivamente a cada iteração.

Este modelo é implementado pelo nó **Gradient Boosted Trees Learner**. Foram testadas combinações de dois parâmetros principais: a profundidade máxima de cada árvore - sem limite ou limitada a 4 níveis - e o número de árvores no ensemble -

100 ou 200. A *learning rate* foi mantida em 0,01 em todas as configurações, o que significa que cada árvore contribui com um passo pequeno para a correção dos erros, tornando o processo mais gradual e estável. Em contextos de *boosting*, árvores mais rasas (profundidade limitada) tendem a funcionar melhor do que árvores completas, porque o objetivo de cada árvore não é resolver o problema sozinha, mas sim corrigir uma pequena parte dos erros do ensemble.



Figura 11: Exemplo de alguns modelos com o nó Gradient Boosted Trees Learner

3.3.5. RProp MLP Learner (Redes Neurais)

Uma rede neuronal artificial é um modelo inspirado no funcionamento do cérebro humano: é constituída por camadas de neurónios artificiais, em que cada neurónio recebe sinais das camadas anteriores, aplica uma transformação e passa o resultado para a camada seguinte. O modelo aprende ajustando os pesos dessas ligações com base nos erros cometidos.

O nó utilizado é o **RProp MLP Learner**, que implementa o algoritmo de treino Resilient Backpropagation (RProp). Este algoritmo adapta individualmente a velocidade de aprendizagem de cada peso da rede ao longo do treino, tornando o processo mais estável.

Antes de aplicar este modelo, os dados foram normalizados com escala *min-max*, um pré-processamento obrigatório para redes neurais baseadas em gradiente: sem normalização, atributos com escalas muito diferentes — como a temperatura em graus e a pressão em hPa — dominam o processo de aprendizagem de forma desequilibrada.

Foram testadas cinco configurações de arquitetura, variando o número de camadas ocultas (5, 10 e 15) e o número de neurónios por camada (5, 10, 15 e 20), com 500 e 1000 iterações de treino. A escolha da arquitetura é um dos aspetos mais desafiantes das redes neuronais: redes com poucas camadas ou poucos neurónios podem não ter capacidade suficiente para aprender padrões complexos, enquanto redes demasiado grandes podem memorizar os dados de treino sem generalizar.

3.3.6. Modelos de Clustering

Os modelos de *clustering* pertencem ao paradigma de aprendizagem não supervisionada: ao contrário dos modelos anteriores, não utilizam a variável alvo durante o treino. O objetivo é encontrar agrupamentos naturais nos dados com base apenas nas características disponíveis, sem saber à partida a que classe cada registo pertence.

Foram utilizados dois algoritmos: **K-Means**, implementado no nó **K-Means** do KNIME, e **K-Medoids**, no nó **K-Medoids**. Ambos foram configurados com 5 *clusters* - correspondendo ao número de classes da variável alvo - e um máximo de 100 iterações. A diferença entre os dois está na forma como cada algoritmo define o centro de cada grupo: o K-Means usa o ponto médio (centróide), que é um valor calculado e pode não corresponder a nenhum registo real; o K-Medoids usa o registo mais central do grupo (*medoid*), que é sempre um ponto real do dataset, tornando-o mais robusto à presença de valores atípicos.

Para avaliar os resultados dos modelos de *clustering* — que, por serem não supervisionados, não produzem diretamente uma previsão da classe — foi utilizada como referência a variável **Consumptions**, comparando a que *cluster* cada registo foi atribuído com a sua classe real. Para facilitar esta comparação, foram também testadas versões com ordenação prévia do dataset pela coluna **Consumptions**, de forma a alinhar a ordem dos *clusters* gerados com a ordem natural das classes de consumo.

3.4. Resultados obtidos e discussão

A análise dos resultados obtidos permite tirar conclusões a vários níveis: sobre o efeito das estratégias de tratamento de dados, sobre o comportamento de cada algoritmo com diferentes parametrizações, e sobre a adequação de cada abordagem ao problema em causa.

A tabela seguinte apresenta os resultados de *accuracy* obtidos pelas configurações mais representativas de cada modelo em cada variante de tratamento, com partição 70/30.

Modelo / Config.	Fl. principal	V. Vento	V. Vol-tage	V. Terres-trial	V. Atm. Me-diana	V. Atm. Merge
DT - Gini, com pruning	93,74%	93,74%	87,01%	87,13%	93,74%	93,74%
DT - Gini, sem pruning	88,73%	88,69%	83,62%	83,47%	89,42%	89,11%
DT - Gain Ratio, com pruning	93,74%	93,74%	84,00%	84,00%	93,74%	93,74%
RF - Gini, 200 modelos	93,70%	93,66%	87,70%	87,55%	93,70%	93,66%
RF - Inf. Gain, 100 modelos	93,66%	93,58%	87,55%	87,13%	93,62%	93,54%

Modelo / Config.	Fl. principal	V. Vento	V. Vol- tage	V. Terres- trial	V. Atm. Me- diana	V. Atm. Merge
LR - IRLS, float, com norm.	84,76%	84,76%	73,61%	73,61%	84,38%	84,80%
LR - SAG, float, com norm.	84,80%	84,57%	73,57%	73,49%	84,64%	84,84%
GBT - depth 4, 200 modelos	93,54%	93,54%	85,06%	85,10%	93,58%	93,54%
GBT - sem limite, 100 modelos	87,59%	87,43%	80,44%	80,29%	87,24%	87,74%
MLP - 1000 it., 5×15	90,64%	89,65%	80,98%	81,67%	90,99%	91,14%
MLP - 500 it., 15×5	38,08%	30,25%	30,98%	30,56%	38,16%	25,06%
K-Means, 5 clusters	65,32%	65,32%	68,95%	68,95%	65,32%	65,32%
K-Medoids, 5 cl., ordenado	78,04%	78,04%	71,77%	71,77%	78,04%	78,04%

3.4.1. Impacto do tratamento de dados nos resultados

A análise comparativa entre variantes permite identificar com clareza quais as decisões de tratamento de dados que mais influenciam o desempenho dos modelos.

As variantes que apresentaram um impacto negativo mais expressivo foram a V. Voltage e a V. Terrestrial. O fator comum entre estas duas variantes é precisamente o mesmo: em ambas, os valores em falta nas cinco colunas de tensão (Very High Voltage, High Voltage, Medium Voltage, Low Voltage e Total) foram substituídos pela mediana global em vez de interpolação linear. Esta escolha provocou uma descida consistente de cerca de 6 a 8 pontos percentuais em todos os modelos supervisionados face ao fluxo principal e às restantes variantes. A título de exemplo, a Decision Tree com *pruning* passou de 93,74% para 87,01% na V. Voltage e para 87,13% na V. Terrestrial; o Random Forest desceu de 93,7% para 87,7% e 87,6%, respetivamente; e o Gradient Boosted Trees de 93,54% para cerca de 85%.

Este resultado demonstra que as colunas de tensão têm um peso considerável no desempenho dos modelos, o que é esperado dado que estão diretamente relacionadas com a variável alvo **Consumptions**. Sendo séries temporais com forte continuidade - os valores de uma hora são tipicamente muito próximos dos da hora anterior e seguinte -, a interpolação linear é uma estratégia de tratamento muito mais adequada do que a mediana global: a interpolação estima cada valor em falta com base nos vizinhos temporais, preservando a estrutura da série, enquanto a mediana substitui todos os valores em falta pela mesma constante, destruindo essa continuidade e introduzindo descontinuidades artificiais.

Importa ainda notar que a V. Terrestrial difere do fluxo principal não apenas na estratégia de tratamento das colunas de tensão (voltage), mas também no tratamento da coluna **terrestrial_radiation**: nesta variante, os valores em falta foram preenchidos com interpolação linear em vez das regras baseadas em conhecimento de domínio utilizadas no fluxo principal. No entanto, esta diferença não acresceu qualquer impacto negativo adicional nos resultados - os valores obtidos na V. Terrestrial são muito próximos dos da V. Voltage para os mesmos modelos, o que confirma que as duas abordagens de tratamento para a coluna **terrestrial_radiation** são aproximadamente equivalentes em termos de qualidade dos dados produzidos. A descida de desempenho

observada nesta variante deve-se exclusivamente ao tratamento das colunas de tensão por mediana.

As restantes variantes - V. Vento, V. Atm. Mediana e V. Atm. Merge - produzem resultados muito próximos do fluxo principal, com diferenças inferiores a 0,2 pontos percentuais na generalidade dos modelos. A variante V. Vento, que usa mediana para todas as colunas de vento, não provoca qualquer degradação visível, o que indica que estas variáveis têm pouco peso no poder discriminativo dos modelos para este problema. As variantes atmosféricas confirmam que tanto a interpolação linear como a mediana são estratégias igualmente possíveis para variáveis como temperatura e humidade, e que a decisão de fundir ou manter separadas as colunas de temperatura e temperatura aparente é indiferente para o desempenho final.

3.4.2. Decision Tree

A Decision Tree com *pruning* MDL obteve os resultados mais estáveis e consistentes de todos os modelos testados. Com o critério Gini Index e com *pruning*, a *accuracy* foi de 93,74% tanto na partição 70/30 como na validação cruzada de 10 iterações (93,72%), nas variantes de tratamento equivalentes ao fluxo principal. O critério Gain Ratio com *pruning* produziu exatamente o mesmo resultado, o que mostra que a escolha do critério de divisão tem pouco peso quando o *pruning* está ativo - o que realmente importa é o processo de poda, não a forma como cada divisão é escolhida.

Sem *pruning*, os resultados caem cerca de 4% a 5%, dependendo do critério e da variante. A queda é maior com o critério Gini do que com o Gain Ratio: sem *pruning*, o Gain Ratio obteve 89,76% no fluxo principal, contra 88,73% do Gini.

O comportamento sem *pruning* é um exemplo clássico de *overfitting*: a árvore cresce até memorizar os dados de treino, incluindo ruído e valores tratados, e perde a capacidade de generalizar. Com *pruning*, esse problema é corrigido, e a árvore aprende apenas os padrões mais robustos e generalizáveis.

A consistência dos resultados entre partição 70/30 e validação cruzada - com diferenças mínimas - confirma que o dataset é suficientemente grande e equilibrado para que uma divisão simples produza avaliações fiáveis.

3.4.3. Random Forest

O Random Forest produziu resultados muito próximos da Decision Tree com *pruning*, com *accuracy* entre 97,13% e 93,74%. A diferença entre usar 100 ou 200 árvores é negligenciável, o que indica que o ensemble já estabiliza com 100 árvores e que adicionar mais não traz um ganho real. Da mesma forma, as diferenças entre os três critérios de divisão testados (Gini, Information Gain e Information Gain Ratio) são mínimas, confirmando a robustez característica do Random Forest relativamente à escolha dos parâmetros.

O facto de o Random Forest não superar a Decision Tree simples com *pruning* neste problema pode parecer surpreendente, mas tem uma explicação plausível: após o tratamento de dados, o dataset é suficientemente limpo e estruturado para que uma única árvore bem podada capture a maior parte dos padrões discriminativos. O *ensemble* de

árvores introduz diversidade, mas quando o problema tem uma estrutura clara e os dados são de boa qualidade, essa diversidade não produz ganho adicional relevante.

3.4.4. Logistic Regression

A regressão logística apresentou os resultados mais baixos entre os modelos supervisionados, com *accuracy* entre 73% e 87% dependendo da configuração e variante. A configuração com apenas campos numéricos (*float*) superou consistentemente a versão com todos os campos: por exemplo, no fluxo principal, a versão *float* com IRLS sem normalização obteve 84,53% contra 83,73% da versão com todos os campos. A razão é que as variáveis categóricas presentes no dataset - como *estacao_ano*, *Diffuse_Radiation* e *Direct_Radiation* - não estão codificadas numericamente e o nó do KNIME não as processa adequadamente, introduzindo ruído no modelo.

A normalização não produziu melhorias expressivas para o método IRLS — as diferenças são inferiores a 1 ponto percentual -, mas é necessária para o método SAG, e requer que todas as variáveis estejam na mesma escala. Sem normalização, o SAG convergeria de forma desequilibrada, dando demasiado peso a atributos com valores numericamente maiores.

O desempenho globalmente mais baixo da regressão logística tem uma explicação estrutural: este é um modelo linear, o que significa que assume que a fronteira entre classes pode ser descrita por combinações lineares dos atributos. No entanto, o consumo energético depende de interações complexas e não lineares entre variáveis — por exemplo, a temperatura elevada tem impacto diferente no consumo consoante a hora do dia e a estação do ano. Essas interações não são capturáveis por um modelo linear.

3.4.5. Gradient Boosted Trees

O Gradient Boosted Trees com profundidade limitada a 4 obteve resultados muito próximos da Decision Tree com *pruning* (entre 85,06% e 93,58%). É o modelo que, em conjunto com a Decision Tree e o Random Forest, apresenta o melhor desempenho global.

A configuração sem limite de profundidade fica claramente abaixo, com valores entre 80% e 88%, o que confirma um princípio fundamental do *gradient boosting*: cada árvore do ensemble deve ser um classificador fraco, que corrige apenas uma parte dos erros, e não um classificador completo. Quando a profundidade não tem limite, cada árvore tende a memorizar os dados de treino em vez de corrigir os erros de forma controlada, o que desequilibra o processo de aprendizagem sequencial. Árvores mais curtas, com profundidade 4, mantêm cada contribuição pequena e focada, e o ensemble converge para uma solução mais robusta.

Tal como no Random Forest, a diferença entre 100 e 200 modelos é pouco expressiva, confirmando que o ensemble estabiliza rapidamente.

3.4.6. Redes Neurais (RProp MLP)

As redes neurais foram o modelo com maior variabilidade de resultados: entre a pior e a melhor configuração testada, a diferença chega a 60 pontos percentuais. Isto reflete a sensibilidade elevada das redes neurais à escolha da arquitetura, que é um dos aspetos mais difíceis de acertar.

A configuração com 500 iterações, 15 camadas ocultas e 5 neurónios por camada obteve sistematicamente os piores resultados de todo o trabalho — entre 25% e 38% consoante a variante. Este resultado tem uma justificação: uma rede muito profunda (muitas camadas ocultas) mas muito estreita (poucas unidades por camada) tem dificuldade em representar padrões complexos porque o sinal se degrada ao longo das camadas sem ter largura suficiente para o transportar.

As melhores configurações foram as de 5 camadas ocultas com 15 ou 20 neurónios por camada e 1000 iterações, que atingiram até 91,14% na variante V. Atm. Merge e 90,99% na variante V. Atm. Mediana. Redes mais largas e menos profundas parecem adaptar-se melhor a este problema - conseguem capturar as relações não lineares sem o problema de degradação do sinal que afeta redes muito profundas.

Apesar de os melhores resultados das redes neuronais ficarem ligeiramente abaixo dos modelos de árvore, a diferença é relativamente pequena.

3.4.7. Clustering

Os modelos de *clustering* não foram treinados para prever diretamente as classes de consumo, mas os seus resultados foram avaliados por comparação com a variável alvo **Consumptions**. O K-Means obteve 65,32% no fluxo principal e 66,04% com ordenação prévia da coluna alvo — uma melhoria pouco significativa. O K-Medoids, por outro lado, reagiu de forma muito mais expressiva à ordenação: sem ordenação obteve 61,60%, mas com ordenação prévia subiu para 78,04%.

Esta diferença de comportamento é interessante: o K-Medoids é mais sensível à inicialização do que o K-Means, porque o seu processo de otimização parte de pontos reais do dataset e é mais suscetível a mínimos locais. A ordenação prévia da coluna **Consumptions** fornece uma âncora que orienta a inicialização do algoritmo, alinhando os clusters com a estrutura ordinal das classes e evitando que o algoritmo convirja para uma solução mal alinhada.

Os resultados do *clustering* são, como esperado, inferiores aos dos modelos supervisionados - a diferença de cerca de 15 a 28 pontos percentuais face à Decision Tree com *pruning* reflete o custo de aprender sem a orientação da variável alvo. Ainda assim, uma *accuracy* de 78% para o K-Medoids com ordenação é um resultado notável para aprendizagem não supervisionada, e sugere que os dados apresentam uma estrutura de agrupamento natural que é coerente com as classes de consumo definidas.

3.4.8. Partição simples vs. validação cruzada

Uma diferença relevante e consistente observada nos resultados é que a validação cruzada (cross-validation) com 10 iterações (X-Partitioner) tende a produzir resultados ligeiramente superiores aos da partição simples 70/30 para os modelos de regressão logística, e muito próximos para os modelos de árvore.

Para a regressão logística, por exemplo, a configuração IRLS com campos *float* e sem normalização obteve 84,53% com partição simples e 86,12% com validação cruzada no fluxo principal - uma diferença de 1,5 pontos percentuais. Esta diferença explica-se pelo facto de a validação cruzada usar, em cada iteração, 90% dos dados para treino (contra

70% na partição simples), o que beneficia especialmente modelos mais simples como a regressão logística, que são mais sensíveis à quantidade de dados disponíveis para treino.

Para os modelos de árvore, a diferença entre as duas abordagens é mínima - geralmente inferior a 0,1 pontos percentuais - porque estes modelos são menos sensíveis à quantidade de dados de treino quando o dataset tem mais de alguns milhares de registos, como é o caso.

A validação cruzada é a abordagem mais rigorosa e menos influenciada pela divisão específica dos dados, mas o custo computacional é 10 vezes superior ao da partição simples. Para modelos rápidos como a Decision Tree, esse custo é negligenciável; para modelos mais lentos como as redes neuronais, pode ser proibitivo, daí não haver dados da aplicação da validação cruzada para modelos de redes neuronais e modelos de *clustering*.

3.4.9. Sumário e melhores resultados

Em termos gerais, os resultados obtidos permitem tirar as seguintes conclusões:

A Decision Tree com pruning MDL, o Random Forest e o Gradient Boosted Trees com profundidade 4 foram os modelos com melhor desempenho, ambos atingindo cerca 93,7% de *accuracy* nas variantes de tratamento com boa qualidade de dados. São também os modelos mais estáveis - os seus resultados variam pouco com a parametrização, desde que as escolhas principais (presença de *pruning* na Decision Tree e profundidade limitada Gradient Boosted) sejam feitas corretamente.

As Redes Neuronais mostraram o maior potencial de melhoria: com a arquitetura certa (5 camadas, 15–20 neurónios, 1000 iterações), chegaram a 91%, mas exigem uma escolha cuidadosa de parâmetros - configurações erradas levam a resultados muito baixos.

A Regressão Logística ficou sistematicamente abaixo dos restantes modelos, o que é esperado dado que o problema tem uma estrutura não linear que um modelo linear não consegue capturar completamente.

O K-Medoids com ordenação foi o melhor modelo não supervisionado com 78,04%, demonstrando que os dados têm uma estrutura de agrupamento natural coerente com as classes de consumo.

A decisão de tratamento de dados com maior impacto foi a estratégia de tratamento das colunas de tensão: a interpolação linear é claramente superior à mediana para estas colunas, com uma diferença acentuada em todos os modelos. Esta é a lição mais importante retirada do estudo de variantes: para séries temporais com forte continuidade, a interpolação linear preserva a estrutura dos dados de forma muito mais fiel do que uma estatística global como a mediana.

4. Dataset de grupo

4.1. Exploração do dataset

O dataset escolhido pelo grupo incide sobre o desempenho académico de vários estudantes, reunindo 6607 registos e 20 colunas (variáveis) que descrevem fatores pessoais, familiares, escolares e comportamentais. Cada registo corresponde a um aluno e inclui informação sobre hábitos de estudo, assiduidade, envolvimento parental, acesso a

recursos, condições socioeconômicas, qualidade do ensino, características individuais e resultados anteriores. A variável alvo, **Exam_Score**, representa o resultado, de 0 a 100, obtido pelo estudante e serve como referência para analisar de que forma estes diferentes fatores contribuem para o seu desempenho global.

A fase de exploração foi feita antes de qualquer intervenção nos dados, com o objetivo de perceber o comportamento de cada variável, identificar problemas de qualidade de dados e perceber as relações existentes entre os campos.

4.1.1. Variáveis discretas

A análise das variáveis discretas evidencia distribuições marcadamente desequilibradas em várias categorias, refletindo padrões estruturais claros no dataset. Variáveis binárias como **School_Type**, **Internet_Access**, **Learning_Disabilities** e **Gender** apresentam forte predominância de uma das classes, o que indica baixa variabilidade e poderá influenciar modelos sensíveis à disparidade. O resto, variáveis com três categorias, todas ordinais, como **Parental_Involvement**, **Motivation_Level**, **Access_to_Resources** e **Distance_from_Home**, revelam também distribuições heterogêneas, com categorias dominantes bem definidas. Adicionalmente, a presença de valores em falta em variáveis como **Teacher_Quality**, **Parental_Education_Level** e **Distance_from_Home** requer tratamento adequado antes de qualquer modelação.

Column	Exclude Column	No. missings	Unique values	All nominal values	Frequency Bar Chart
Access_to_Resources	☐	0	3	Medium, High, Low	
Distance_from_Home	☐	67	3	Near, Moderate, Far	
Extracurricular_Activities	☐	0	2	Yes, No	
Family_Income	☐	0	3	Low, Medium, High	
Gender	☐	0	2	Male, Female	
Internet_Access	☐	0	2	Yes, No	
Learning_Disabilities	☐	0	2	No, Yes	

Figura 12: Análise das variáveis discretas (parte 1)

Column	Exclude Column	No. missings	Unique values	All nominal values	Frequency Bar Chart
Teacher_Quality	☐	78	3	Medium, High, Low	
School_Type	☐	0	2	Public, Private	
Peer_Influence	☐	0	3	Positive, Neutral, Negative	
Parental_Involvement	☐	0	3	Medium, High, Low	
Parental_Education_Level	☐	90	3	High School, College, Postgraduate	
Motivation_Level	☐	0	3	Medium, Low, High	

Figura 13: Análise das variáveis discretas (parte 2)

4.1.2. Variáveis contínuas

A análise das variáveis contínuas revela um conjunto de padrões que ajudam a caracterizar o comportamento geral dos estudantes no dataset. Variáveis como **Hours_Studied**, **Attendance**, **Sleep_Hours** e **Previous_Scores** apresentam distribuições relativamente equilibradas, com valores médios consistentes e baixa assimetria, sugerindo hábitos e desempenhos estáveis na maioria dos alunos. Em contraste, **Tutoring_Sessions** evidencia uma distribuição mais assimétrica, concentrada em valores baixos, indicando que apenas uma parte reduzida dos estudantes recorre regularmente a apoio adicional. Já **Physical_Activity** mostra uma dispersão moderada, refletindo diversidade nos níveis de atividade semanal. **Exam_Score** destaca-se por apresentar assimetria positiva e elevada curtose, revelando uma concentração de resultados próximos da média, mas com alguns valores mais altos que estendem a cauda da distribuição. Salta à vista, também na variável alvo, o valor máximo, 101, que ultrapassa o limite esperado dos resultados

sugerindo a presença de um outlier ou de um erro de registo a considerar na limpeza dos dados.

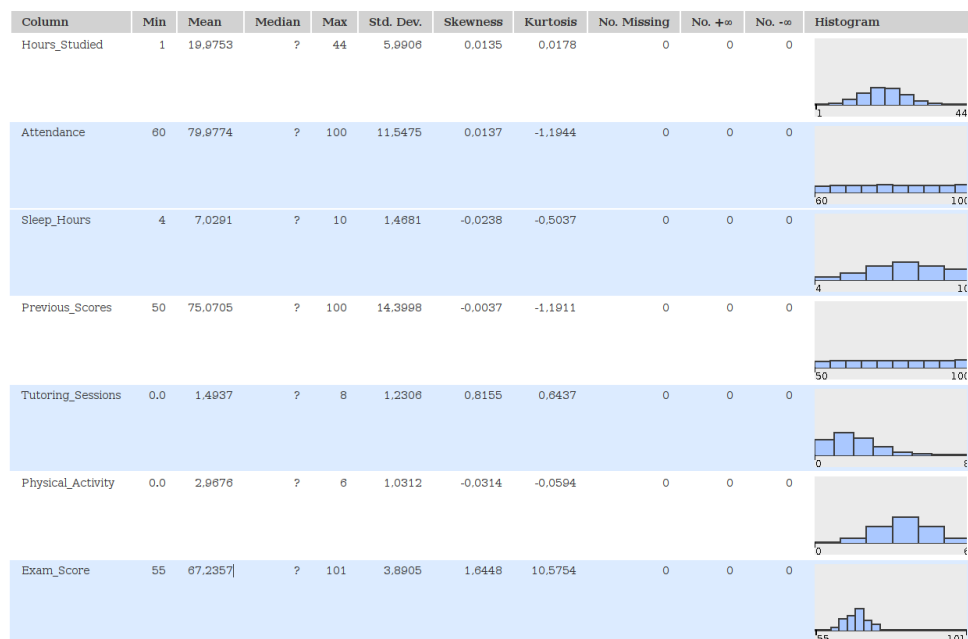


Figura 14: Análise estatística das variáveis contínuas

4.1.3. Correlação entre variáveis

A matriz de **Rank Correlation** evidencia de forma clara quais as variáveis com maior relação com o Exam_Score. Fatores académicos diretamente ligados ao desempenho, como Hours_Studied, Previous_Scores e Attendance, apresentam as correlações mais elevadas, reforçando a sua relevância na previsão de resultados. Em contraste, várias variáveis discretas, como Gender e School_Type, surgem com correlações muito próximas de zero, indicando ausência de relação com a variável alvo. Esta constatação levou, na fase do tratamento dos dados, à criação de fluxos alternativos: uns que incluem todas as variáveis e outros que filtram estas categorias, permitindo avaliar se a sua presença afeta de alguma forma as previsões.

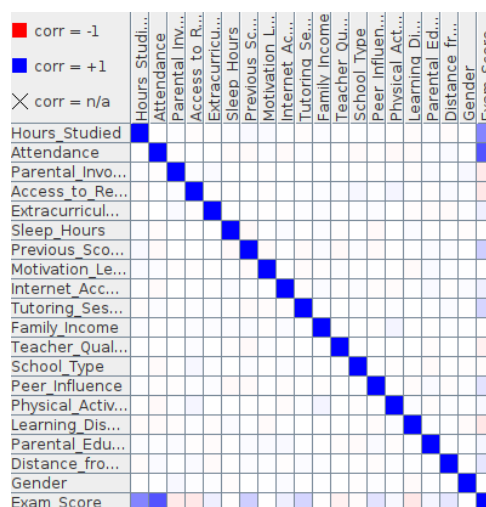


Figura 15: Correlação entre variáveis

A análise dos **density plots** confirma visualmente aquilo que a matriz sugeria: tanto **Gender** como **School_Type** apresentam distribuições de **Exam_Score** praticamente sobrepostas entre categorias, exibindo picos nas mesmas regiões e formas muito semelhantes, sem diferenças sistemáticas que indiquem impacto no desempenho.

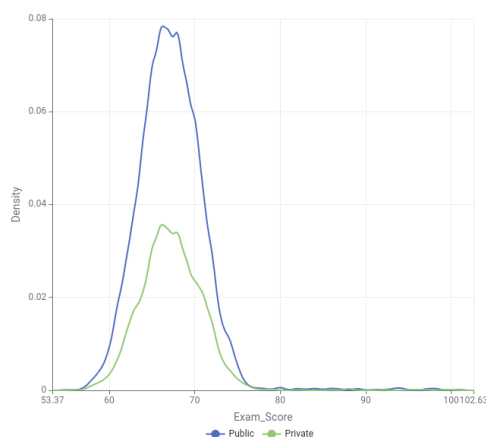


Figura 16: Distribuição de Exam_Score em função de School_Type

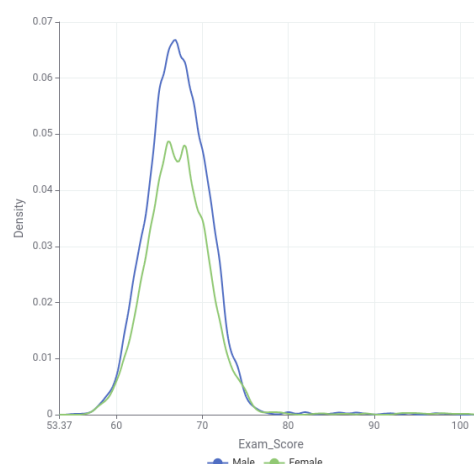


Figura 17: Distribuição de Exam_Score em função de Gender

4.2. Tratamento de dados

A fase de tratamento de dados é essencial para garantir que toda a informação usada nos modelos está limpa, consistente e preparada para ser interpretada corretamente pelos algoritmos. É aqui que se resolvem problemas como valores inválidos, dados em falta, escalas incompatíveis ou variáveis que precisam de ser transformadas para formatos numéricos. Esta etapa não melhora apenas a qualidade dos dados, melhora a qualidade dos próprios modelos, reduzindo ruído, evitando enviesamentos e assegurando que as comparações entre diferentes abordagens de machine learning são justas e fiáveis.

O tratamento iniciou-se com a aplicação de **Row Filter** para garantir a integridade da variável alvo, removendo registos cujo **Exam_Score** se encontra fora da escala (0–100). Deste nó resulta apenas um remoção. Seguem-se duas abordagens alternativas para lidar com valores em falta: uma que procede à imputação através do **Most Frequent Value**, preservando o máximo de informação possível, e outra que opta por remover os 229 registos incompletos, assegurando que apenas observações inteiras são utilizadas. Adicionalmente, são testadas outras duas variantes, uma que mantém todas as variáveis e outra que remove os atributos **Gender** e **School_Type** através de um **Column Filter**, decisão fundamentada na fase anterior que revelou a ausência de correlação relevante destas variáveis com a variável alvo. Após esta filtragem, procede-se à transformação das variáveis discretas: categorias com mais que duas opções são convertidas em valores numéricos ordenados através de **Category to Number**, enquanto que as variáveis binárias são expandidas em colunas indicadoras (**Yes_<variável>** e **No_<variável>**) via **One to Many**, garantindo compatibilidade com algoritmos de regressão. Por fim, são consideradas duas variantes de normalização, aplicando ou não **Z-score** às variáveis contínuas, exceto a variável alvo, permitindo avaliar o impacto da escala dos atributos no desempenho dos modelos.

A combinação destas decisões origina **oito fluxos alternativos**, resultantes da combinação das três bifurcações principais: método de tratamento de missing values, inclusão ou remoção de variáveis discretas sem correlação significativa e normalização das variáveis contínuas. Esta estrutura experimental permite avaliar como diferentes estratégias de pré-processamento influenciam o comportamento dos modelos de machine learning implementados na fase seguinte.

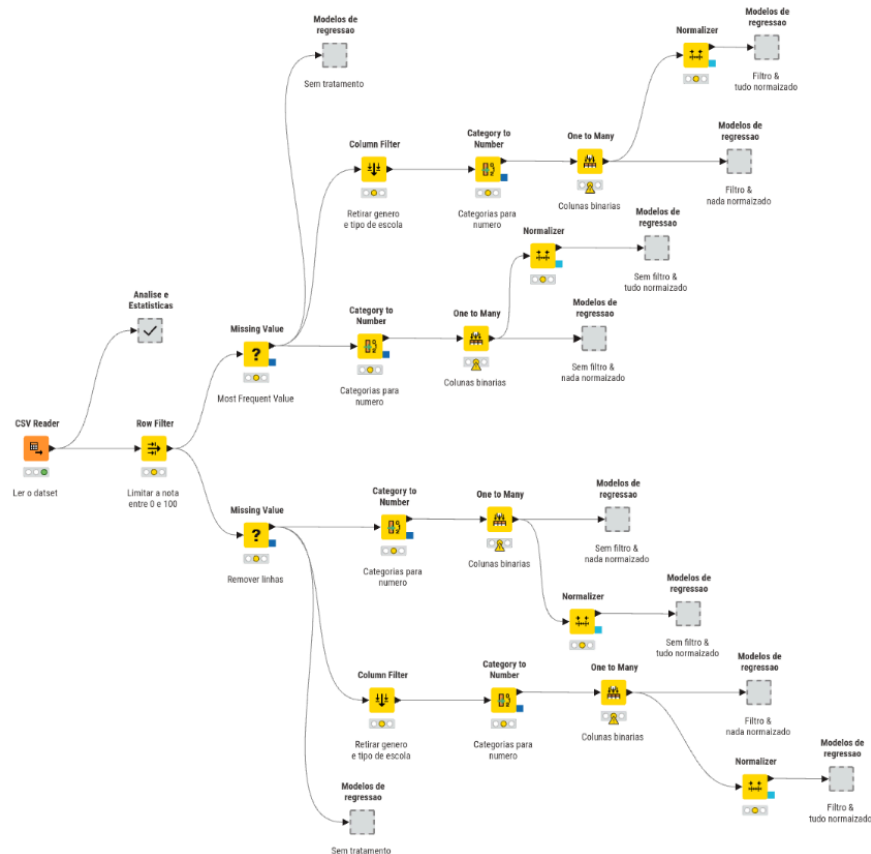


Figura 18: Vista geral do workflow

4.3. Modelos desenvolvidos

A fase de desenvolvimento dos modelos consiste em aplicar diferentes algoritmos de regressão aos fluxos de dados previamente preparados, avaliando como cada combinação de pré-processamento influencia o desempenho preditivo. O objetivo não é apenas comparar modelos entre si, mas compreender como a complexidade, a capacidade de generalização e a sensibilidade aos dados variam entre abordagens lineares, polinomiais, redes neurais e métodos baseados em árvores.

4.3.1. Validação

Nesta fase, cada um dos oito fluxos de dados origina duas variantes adicionais porque são avaliados com dois métodos de validação distintos, duplicando assim o número total de execuções. A primeira abordagem utiliza o **Table Partitioner**, que implementa uma validação hold-out 70/30, onde 70% dos dados são usados para treinar o modelo e os

restantes 30% para o testar. Este método é simples e rápido, mas depende fortemente da forma como os dados são divididos numa única partição. A segunda abordagem recorre ao **X-Partitioner** em conjunto com o **X-Agregator**, permitindo realizar validação cruzada, onde os dados são divididos em 20 subconjuntos (folds) e cada um é usado alternadamente como conjunto de teste. Este método reduz significativamente o risco de **overfitting**, porque impede que o modelo se ajuste demasiado a uma única divisão dos dados e obriga-o a manter desempenho consistente em múltiplos subconjuntos. Assim, a utilização de ambos os métodos assegura uma avaliação mais completa, equilibrada e fiável dos modelos desenvolvidos.

Fluxo	Validação	Missing Values	Gender e School_Type	Normalização
1	Cruzada	Most Frequent Value	Mantidos	Não
2	Cruzada	Most Frequent Value	Mantidos	Z-score
3	Cruzada	Most Frequent Value	Filtrados	Não
4	Cruzada	Most Frequent Value	Filtrados	Z-score
5	Cruzada	Remove Row	Mantidos	Não
6	Cruzada	Remove Row	Mantidos	Z-score
7	Cruzada	Remove Row	Filtrados	Não
8	Cruzada	Remove Row	Filtrados	Z-score
9	Hold-out	Most Frequent Value	Mantidos	Não
10	Hold-out	Most Frequent Value	Mantidos	Z-score
11	Hold-out	Most Frequent Value	Filtrados	Não
12	Hold-out	Most Frequent Value	Filtrados	Z-score
13	Hold-out	Remove Row	Mantidos	Não
14	Hold-out	Remove Row	Mantidos	Z-score
15	Hold-out	Remove Row	Filtrados	Não
16	Hold-out	Remove Row	Filtrados	Z-score

4.3.2. Linear Regression

A regressão linear é o modelo mais simples e interpretável entre os escolhidos, pois assume que a relação entre **Exam_Score** e as outras as variáveis pode ser descrita por uma combinação linear dos atributos. É útil como base porque estabelece o desempenho mínimo esperado sem recorrer a modelos complexos. No KNIME, este método foi implementado com o nó **Linear Regression Learner**, onde o modelo ajusta automaticamente os coeficientes através de mínimos quadrados. A configuração é simples: define-se **Exam_Score** como a variável alvo e selecionam-se as restantes colunas para serem as variáveis independentes, e o nó produz o modelo sem necessidade de ajustes adicionais.

4.3.3. Polynomial Regression

A regressão polinomial é uma extensão direta da regressão linear que permite modelar relações não lineares ao introduzir termos polinomiais dos atributos originais. Em vez de assumir uma relação estritamente linear, o modelo incorpora curvaturas e padrões mais complexos, mantendo ainda assim uma estrutura matemática relativamente simples e interpretável. No KNIME, este método foi implementado através do nó **Polynomial Regression Learner**, que gera automaticamente os termos polinomiais necessários e ajusta o modelo numa única etapa. Configuraram-se 2 modelos com diferente grau polinomial máximo: um quadrado (grau 2) e um cúbico (grau 3). Os graus 2 e 3 permitem testar diferentes níveis de flexibilidade: o grau 2 captura curvaturas suaves, enquanto o grau 3 permite modelar variações mais acentuadas, embora com maior risco de sobreajuste (overfitting).

4.3.4. Random Forest

A Random Forest é um método baseado em bagging que combina múltiplas árvores de decisão treinadas sobre subconjuntos aleatórios dos dados de teste e dos atributos, reduzindo variância e aumentando robustez. É particularmente eficaz em capturar interações complexas entre variáveis e em lidar com datasets heterogêneos, sendo menos sensível a ruído e a outliers do que modelos lineares. No KNIME, este método foi implementado com o nó **Random Forest Learner (Regression)**, onde o parâmetro principal ajustado foi o número de árvores, tendo sido criadas duas configurações diferentes com 200 e 500 árvores, respetivamente, permitindo avaliar o impacto da dimensão da floresta na estabilidade das previsões: florestas maiores tendem a produzir resultados mais estáveis, mas com maior custo computacional. A profundidade das árvores, o número mínimo de amostras por nó e os restantes parâmetros foram mantidos nos valores padrão.

4.3.5. Gradient Boosted Trees

As Gradient Boosted Trees constroem o modelo de forma sequencial, cada árvore tentando corrigir os erros da anterior, o que permite capturar padrões complexos com elevada precisão. Este método é sensível aos parâmetros, mas quando bem configurado pode alcançar ótimo desempenho, especialmente em datasets com relações não lineares e interações subtis. No KNIME, o modelo foi implementado com o nó **Gradient Boosted Trees Learner (Regression)**, configurado com profundidade máxima de 5 para controlar a complexidade individual das árvores. As duas variantes testadas diferem no número de modelos (árvores) e no learning rate, permitindo comparar um boosting mais agressivo (100 árvores e LR 0.1) com um processo mais gradual e estável (200 árvores e LR 0.05). Esta configuração permite avaliar o equilíbrio entre rapidez de aprendizagem e capacidade de generalização.

4.3.6. Redes Neurais Artificiais

As redes neurais **MultiLayer Perceptron** (MLP) são modelos capazes de aprender relações altamente não lineares entre variáveis, recorrendo a uma estrutura composta por neurónios organizados em camadas que procuram replicar o funcionamento do cérebro humano. A profundidade e a largura da rede determinam a capacidade sua expressividade: arquiteturas mais profundas conseguem aproximar funções mais complexas, mas também aumentam o risco de sobreajuste e exigem mais dados e mais

iterações de treino para convergir. O algoritmo **Resilient Backpropagation** (RProp) é uma variante do gradiente que ignora a magnitude do gradiente e utiliza apenas o seu sinal, ajustando os pesos com passos adaptativos. Isto torna o treino mais estável e menos sensível à escala dos dados, especialmente útil em datasets heterogêneos como este.

No KNIME, as redes neurais foram implementadas com o nó **RProp MLP Learner**, que permite definir explicitamente a arquitetura e os parâmetros de treino. Foram criadas seis variantes que diferem em três dimensões:

- número de iterações (100 ou 200): controla quanto tempo a rede treina;
- número de camadas escondidas (1 ou 5): determina a profundidade da rede e a sua capacidade de modelar relações complexas;
- número de neurónios por camada (10 ou 15): define a largura da rede e a granularidade com que consegue representar padrões.

As redes mais simples (1 camada de 10 neurónios) funcionam como modelos não lineares leves, rápidos e com baixo risco de sobreajuste, enquanto as redes mais profundas (5 camadas) e mais largas (15 neurónios) exploram o limite superior da capacidade expressiva, permitindo avaliar até que ponto o dataset beneficia de modelos mais complexos. As versões com 200 iterações testam se o aumento do tempo de treino melhora a convergência ou se conduz a sobreajuste. Em conjunto, estas configurações permitem mapear de forma sistemática o comportamento das redes neurais em diferentes níveis de complexidade, fornecendo uma análise completa da sua capacidade de generalização.

4.4. Discussão dos resultados obtidos

4.4.1. Melhores resultados

Modelo/Configuração	Fluxo	R ²	MAE	MSE	MSD
Linear Regression	3	0.717	0.616	4.242	-0.022
Polynomial Regression/2º grau	3	0.729	0.473	4.061	-0.046
Polynomial Regression/3º grau	3	0.727	0.5	4.078	-0.021
Random Forest/200 modelos	4	0.65	1.073	5.23	0.044
Random Forest/500 modelos	4	0.653	1.064	5.19	0.041
GBT/profundidade 5, 100 modelos, LR 0.1	1	0.71	0.668	4.34	-0.152
GBT/profundidade 5, 200 modelos, LR 0.05	3	0.71	0.664	4.335	-0.153
RNA/100 iterações, 1 camada, 10 neurónios	4	0.706	0.717	4.399	0.012
RNA/200 iterações, 1 camada, 10 neurónios	4	0.709	0.667	4.36	0.001
RNA/100 iterações, 1 camada, 15 neurónios	4	0.699	0.743	4.502	0.009
RNA/200 iterações, 1 camada, 15 neurónios	4	0.704	0.724	4.422	0.002
RNA/100 iterações, 5 camadas, 10 neurónios	4	0.702	0.75	4.465	0.013
RNA/200 iterações, 5 camadas, 10 neurónios	4	0.706	0.667	4.403	-0

4.4.2. Piores resultados

Modelo/Configuração	Fluxo	R ²	MAE	MSE	MSD
Linear Regression	16	0.665	0.637	4.897	−0.072
Polynomial Regression/2º grau	15	0.68	0.493	4.687	−0.088
Polynomial Regression/3º grau	15	0.679	0.564	4.703	0.058
Random Forest/200 modelos	13	0.601	1.184	5.972	0.044
Random Forest/500 modelos	13	0.605	1.164	5.908	0.01
GBT/profundidade 5, 100 modelos, LR 0.1	16	0.657	0.712	5.013	−0.155
GBT/profundidade 5, 200 modelos, LR 0.05	13	0.657	0.701	5.013	−0.157
RNA/100 iterações, 1 camada, 10 neurónios	15	0.069	2.669	13.622	−0.139
RNA/200 iterações, 1 camada, 10 neurónios	9	0.463	1.717	8.378	−0.104
RNA/100 iterações, 1 camada, 15 neurónios	15	0.319	1.092	9.96	−0.042
RNA/200 iterações, 1 camada, 15 neurónios	11	0.506	1.574	7.696	0.017
RNA/100 iterações, 5 camadas, 10 neurónios	15	0.252	2.27	10.945	0.033
RNA/200 iterações, 5 camadas, 10 neurónios	11	0.452	1.727	8.542	−0.106

4.4.3. Comparação entre modelos

A análise dos resultados revela diferenças claras entre os vários métodos de regressão aplicados, tanto em termos de desempenho médio como de estabilidade face às alterações no pré-processamento e na validação. Os modelos **lineares** e **polinomiais** destacam-se pela sua consistência, independentemente da combinação de tratamento de dados, estes métodos mantêm valores de R² relativamente elevados e estáveis. Esta estabilidade é particularmente evidente quando se observa que, mesmo com diferentes estratégias de imputação, normalização ou filtragem de variáveis, as variações no erro são pequenas. A regressão linear, por exemplo, atinge R² de 0.687 no melhor caso com hold-out e sobe para 0.717 com validação cruzada, demonstrando que a estrutura dos dados é suficientemente regular para ser captada por modelos simples.

Em contraste, as **redes neuronais MLP** apresentam um comportamento muito mais irregular. Embora algumas configurações atinjam valores competitivos em determinados fluxos com validação cruzada, outras combinações degradam drasticamente o desempenho. Esta amplitude de resultados mostra que as MLP são altamente sensíveis ao pré-processamento e à arquitetura escolhida, e que o dataset disponível não é suficientemente grande para suportar redes profundas ou largas sem perda de generalização.

Os métodos baseados em **árvores** apresentam um comportamento intermédio. Random Forest produz resultados estáveis mas nunca atinge os valores mais elevados observados nos modelos lineares ou polinomiais. Os melhores R² situam-se em torno de 0.65, e a diferença entre 200 e 500 árvores é praticamente nula, indicando que o aumento da floresta não traz ganhos significativos. Já Gradient Boosted Trees aproximam-se mais

dos modelos lineares e polinomiais, atingindo R^2 de 0.709 e 0.71 com validação cruzada. No entanto, também aqui se observa alguma sensibilidade ao tratamento dos dados.

Em geral, a comparação global mostra que os métodos lineares e polinomiais são os mais adequados ao tipo de dados disponível, oferecendo uma combinação de desempenho elevado e estabilidade, surgindo os métodos baseados em árvores como alternativas competitivas. No entanto, as redes neurais, apesar do seu potencial teórico, revelam-se demasiado instáveis para este contexto.

4.4.4. Influência da parametrização dos modelos

Os parâmetros internos de cada modelo mostraram ter um impacto muito distinto no desempenho, variando significativamente entre métodos. Nos modelos polinomiais, o único parâmetro configurável é o **grau polinomial máximo** e, entre o grau 2 e o grau 3, R^2 mantém-se estável, demonstrando que estes modelos captam bem a estrutura dos dados independentemente da complexidade adicional introduzida pelos termos polinomiais.

Nos métodos baseados em árvores, os parâmetros também revelam impacto limitado. Em Random Forest, aumentar o número de **modelos** de 200 para 500 não produz melhorias significativas, os valores de R^2 mantêm-se entre 0.60 e 0.65, e as métricas de erro variam apenas marginalmente. Isto indica que a variância do modelo já se encontra suficientemente reduzida com 200 árvores, e que o dataset não beneficia de florestas maiores. Nas Gradient Boosted Trees, os parâmetros têm um papel mais visível, a configuração com 100 árvores e **learning rate** 0.1 apresenta resultados muito próximos da versão com 200 árvores e learning rate 0.05, embora esta última seja ligeiramente mais estável, o que sugere que o boosting é eficaz, mas não excessivamente sensível ao número de modelos ou à taxa de aprendizagem dentro das gamas testadas.

É nas redes neurais MLP que os parâmetros internos têm o impacto mais pronunciado. A combinação entre número de **iterações**, número de **camadas** e número de **neurónios** produz variações muito amplas no desempenho, desde resultados competitivos até valores extremamente baixos. As redes profundas (5 camadas) apresentam consistentemente pior desempenho do que as redes simples com apenas 1 camada, independentemente do número de iterações, o que indica que o dataset não tem dimensão suficiente para suportar modelos de elevada capacidade. O aumento do número de neurónios de 10 para 15 também não traz benefícios claros e, em vários casos, conduz a maior instabilidade. Mesmo o aumento de iterações de 100 para 200 não garante melhorias: em alguns fluxos melhora ligeiramente o R^2 , mas noutros conduz a sobreajuste e degradação do erro.

4.4.5. Influência do tratamento de dados

O tratamento dos dados revelou ter um impacto significativo no desempenho dos modelos, embora a magnitude e a direção desse impacto variem substancialmente entre métodos. A primeira dimensão analisada foi a estratégia de **tratamento de valores em falta**, comparando imputação pelo valor mais frequente com a remoção de registos. Os resultados mostram de forma consistente que a imputação é superior à eliminação de linhas. Na regressão linear, por exemplo, o melhor desempenho com imputação atinge R^2 de 0.717, enquanto a remoção de linhas reduz este valor para 0.710. Esta tendência repete-se nos modelos polinomiais, onde a imputação permite alcançar R^2 de 0.697

no hold-out e até 0.728 na validação cruzada, ao passo que a eliminação de registos conduz a ligeiras quebras. Nos métodos baseados em árvores, o efeito é semelhante: no Random Forest, a imputação produz R^2 de 0.626 no melhor caso, enquanto a remoção de linhas raramente ultrapassa 0.623. A diferença torna-se ainda mais evidente nas redes neuronais, onde a remoção de registos agrava a instabilidade e conduz a quedas abruptas de desempenho, como se observa em vários fluxos onde o R^2 desce para valores inferiores a 0.10 quando combinada com ausência de normalização. Estes resultados confirmam que, num dataset de dimensão moderada, preservar o máximo de informação é crucial para evitar perda de capacidade preditiva.

A segunda dimensão analisada foi a **remoção das variáveis** Gender e School_Type. A hipótese inicial de que estas variáveis poderiam introduzir ruído ou enviesamento não se confirmou. Pelo contrário, a sua remoção tende a degradar ligeiramente o desempenho em praticamente todos os métodos. Na regressão linear, por exemplo, a remoção destas variáveis reduz o R^2 de 0.687 para 0.685 no hold-out e de 0.717 para 0.716 na validação cruzada. Embora estas diferenças sejam pequenas, tornam-se mais pronunciadas em modelos sensíveis à estrutura dos dados, como as redes neuronais. Em vários fluxos das MLP, a remoção destas variáveis contribui para quedas substanciais no desempenho, especialmente quando combinada com ausência de normalização. Nos métodos baseados em árvores, o impacto é mais discreto, mas ainda assim consistente: tanto no Random Forest como nos Gradient Boosted Trees, os melhores resultados surgem sempre quando Gender e School_Type são mantidos. Isto sugere que, mesmo que o seu contributo individual seja modesto, estas variáveis capturam informação útil sobre o contexto escolar e social dos alunos, contribuindo para reduzir o erro preditivo.

A **normalização dos dados** revelou ser um fator crítico, mas apenas para alguns métodos. Nos modelos lineares e polinomiais, o impacto da normalização é praticamente nulo: os valores de R^2 e as métricas de erro mantêm-se quase inalterados entre versões normalizadas e não normalizadas. Nos métodos baseados em árvores, o efeito é igualmente irrelevante, como seria de esperar, dado que árvores de decisão não dependem da escala dos atributos. No entanto, nas redes neuronais MLP, a normalização é determinante. Em vários fluxos, a ausência de normalização conduz a quebras drásticas de desempenho, com R^2 a descer de valores próximos de 0.70 para valores tão baixos como 0.069, 0.107 ou 0.252. A normalização estabiliza o processo de treino, reduz a sensibilidade aos gradientes e permite que as MLP atinjam desempenhos competitivos, especialmente quando combinada com validação cruzada. Este comportamento confirma que, para métodos baseados em gradientes, a normalização não é apenas recomendável, é essencial.

Por fim, a escolha do **método de validação** tem um impacto transversal e significativo em todos os modelos. A validação hold-out produz resultados razoáveis, mas a validação cruzada melhora sistematicamente o desempenho e reduz a variabilidade entre fluxos. Na regressão linear, a validação cruzada aumenta o R^2 de 0.687 para 0.717; na regressão polinomial, de 0.696 para 0.728; nos Gradient Boosted Trees, de 0.669 para 0.709; e mesmo nas redes neuronais, onde a instabilidade é maior, a validação cruzada suaviza oscilações e melhora o desempenho médio. Esta melhoria generalizada indica que o dataset tem dimensão limitada e beneficia claramente de uma avaliação mais robusta,

que reduz o impacto de divisões desfavoráveis dos dados e fornece estimativas mais fiáveis da capacidade de generalização dos modelos.

No conjunto, os resultados mostram que o tratamento dos dados desempenha um papel central na qualidade das previsões. A imputação de valores em falta, a manutenção de variáveis informativas, a normalização seletiva e a validação cruzada contribuem de forma decisiva para melhorar o desempenho e a estabilidade dos modelos. Estas conclusões reforçam a importância de uma abordagem cuidadosa ao pré-processamento, especialmente em contextos onde o dataset é limitado e os modelos apresentam sensibilidades distintas às transformações aplicadas.

5. Conclusão

Com o desenvolvimento deste trabalho prático, fomos capazes de abordar diversos temas e tópicos falados nas aulas teóricas ao longo deste semestre, tais como, paradigmas de aprendizagem, metodologias de análise de dados, problemas de classificação, problemas de regressão, entre outros. Fomos ainda capazes de colocar em prática, numa escala um pouco maior, as diversas técnicas de exploração e tratamento de dados e criação de modelos, aprendidas nas aulas práticas, mais uma vez com recurso à plataforma KNIME.

Após o término do trabalho, consideramos que conseguimos demonstrar na prática o conhecimento adquirido ao longo do semestre. Em ambos os datasets, percorremos todas as etapas da metodologia SEMMA - desde a exploração inicial dos dados, passando pelo tratamento e preparação, até à construção, parametrização e avaliação dos modelos - o que nos permitiu consolidar uma visão completa de um projeto de Machine Learning, desde os dados em bruto até às conclusões finais.

Ao longo do trabalho foram abordados problemas de classificação e de regressão, enquadrados no paradigma de aprendizagem supervisionada, bem como modelos de *clustering*, no âmbito da aprendizagem não supervisionada. Esta diversidade de abordagens permitiu-nos comparar diferentes paradigmas e compreender as suas diferenças, limitações e contextos de aplicação mais adequados.

De uma forma geral, este trabalho permitiu-nos perceber que o sucesso de um projeto de Machine Learning não depende apenas da escolha dos algoritmos, mas em grande medida da qualidade do trabalho feito antes da modelação - na exploração, no tratamento e na preparação dos dados. Aprendemos também que não existe um modelo universalmente superior: a adequação de cada algoritmo depende das características do problema, da estrutura dos dados e dos objetivos definidos.